Fundamentals and the Runtime

By Wesam Abousaid

Learn what Node.js is, how the runtime works, and why it powers modern JavaScript backends. We'll cover the core concepts, execution model, and the foundational tools you need to start building with confidence.

Table of Contents

1. Why Node.js Exists
2. Node Runtime Architecture
3. Running JavaScript with Node
4. Node Module System (CommonJS)
5. Built-in Node Modules
6. Blocking vs Non-Blocking Code
7. The Callback Pattern
8. The Event Loop
9. NVM — Node Version Manager
10. NPM — Node Package Manager
11. Quick Debugging Reference
12. Bridge to Day 2



01

Why Node.js Exists

The Traditional Server Model

Before Node.js, JavaScript only ran in browsers. Server-side applications were built with Java, PHP, Python — each following a thread-per-request model:

1. Each incoming request spawns a new thread
2. Threads block while waiting for I/O (database, file, network)
3. High memory usage and context-switching overhead
4. Poor scalability under concurrent load

The Node.js Solution

Node.js addressed these limitations with:

- **Single-threaded architecture** — one JS thread for all requests
- **Asynchronous operations** — I/O doesn't block the thread
- **Event-driven design** — responses are handled via callbacks/events

Benefits

- Handles thousands of concurrent connections efficiently
- Ideal for I/O-heavy workloads (APIs, real-time apps, streaming)
- Lightweight memory footprint

Mental Model: Traditional Server

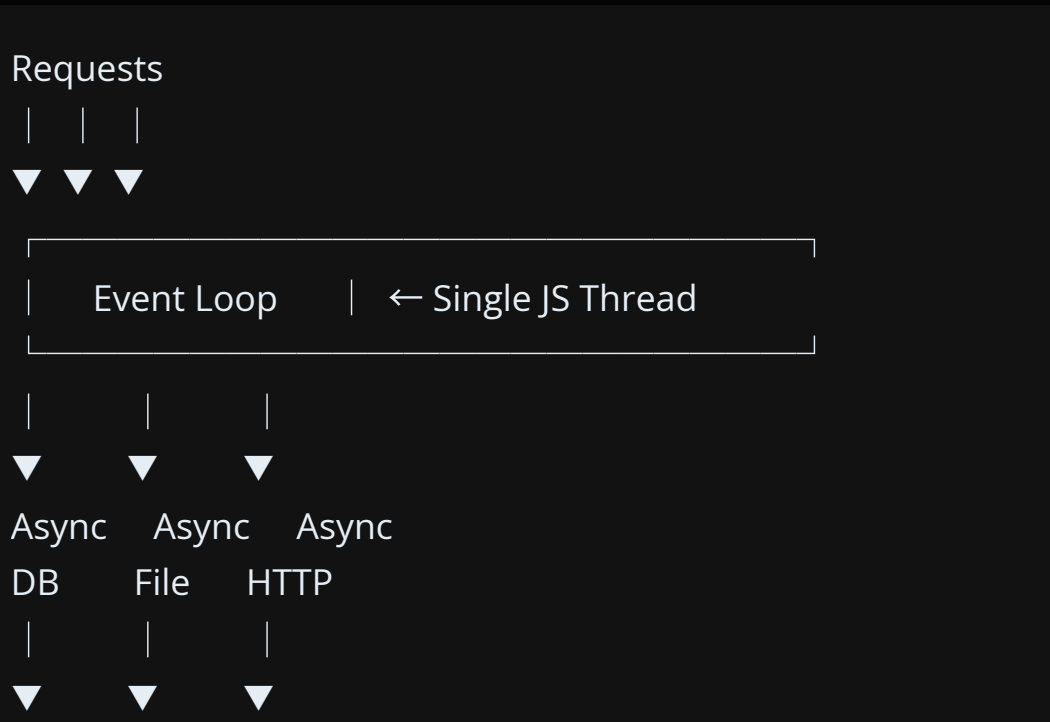
Thread per Request

```
Request 1 ── Thread ── Blocking DB call ── Wait ──► Response
Request 2 ── Thread ── Blocking File I/O ─ Wait ──► Response
Request 3 ── Thread ── Blocking API call ─ Wait ──► Response
```

Result: Many threads, high memory, context switching overhead

Mental Model: Node.js

Event Loop Model



Result: Few threads, non-blocking, high concurrency

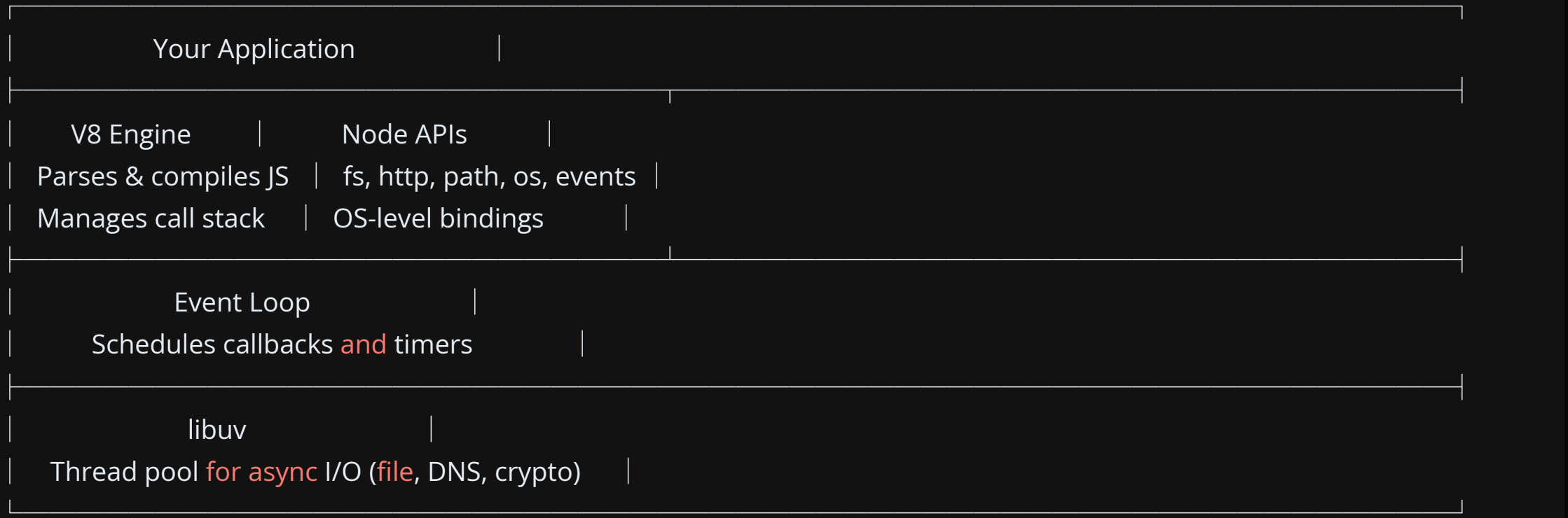


02

Node Runtime Architecture

Node Runtime Architecture

Node.js is not just a JavaScript engine — it's a runtime environment built from several cooperating components:



V8 Engine & Node APIs

V8 Engine

Google's high-performance JavaScript engine (also powers Chrome):

- Call Stack — where synchronous JS executes, frame by frame
- Heap — memory allocation for objects and closures
- Parses JS → compiles to optimised machine code → executes

Node APIs

Capabilities V8 alone doesn't have:

- File system access (fs)
- Networking (http, net)
- OS integration (os, process)
- Module system (require)

Event Loop & libuv

Event Loop

The heart of Node's concurrency model:

- Continuously checks: Is the call stack empty? Are there pending callbacks?
- Moves ready callbacks from the callback queue onto the call stack
- Enables non-blocking execution without multiple threads

libuv

A C library that powers Node's async capabilities:

- Maintains a thread pool (default: 4 threads) for expensive operations
- Offloads: file system access, DNS lookups, compression, crypto
- Networking (TCP/UDP) uses the OS event notification system (epoll/kqueue/IOCP), not the thread pool



03

Running JavaScript with Node

Your First Node Program

Hello, Node

```
// hello.js
console.log("Hello from Node");
```

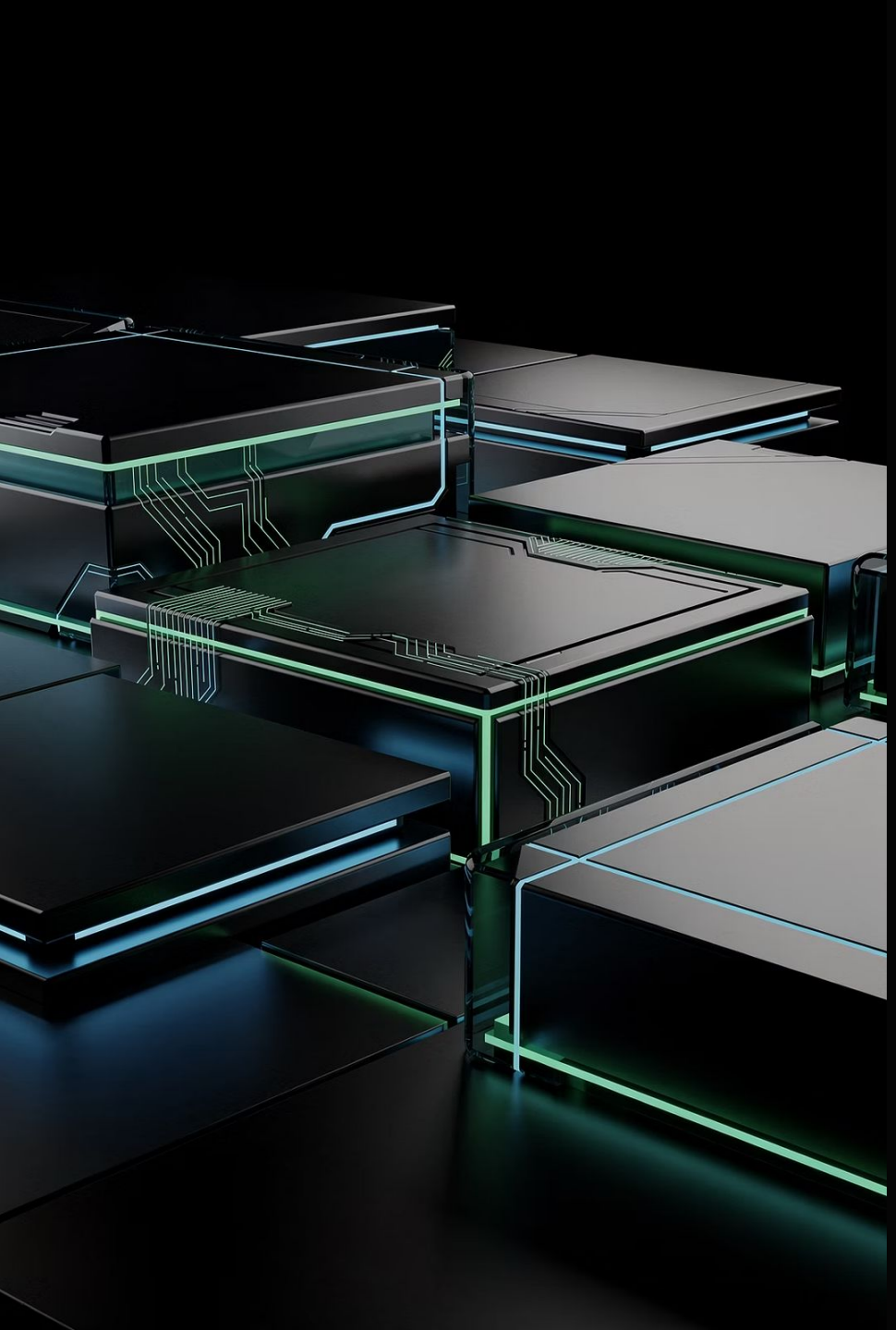
```
node hello.js
# Output: Hello from Node
```

Accessing Process Information

```
// process is a global object — no require needed
console.log(process.version);      // Node.js version, e.g. v20.11.0
console.log(process.platform);     // 'linux', 'darwin', 'win32'
console.log(process.argv);         // Array of CLI arguments
console.log(process.env.HOME);     // Environment variables
console.log(process.cwd());        // Current working directory
```

Key Difference: Browser JS vs Node JS

Feature	Browser JavaScript	Node.js
Primary environment	DOM / Web APIs	OS, file system, network
Global object	window	global / globalThis
Module system	ES Modules (import)	CommonJS (require) + ESM
I/O	Fetch, XHR	fs, http, net
Entry point	HTML <script> tag	node filename.js



04

Node Module System (CommonJS)

CommonJS: Exporting & Importing

Each file in Node is treated as its own module. Modules export values and import from others using require / module.exports.

Exporting

```
// math.js
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

module.exports = { add, subtract };
```

Importing

```
// app.js
const math = require("./math");   // relative path to your file
console.log(math.add(2, 3));      // 5
console.log(math.subtract(10, 4)); // 6
```

```
node app.js
# Output:
# 5
# 6
```

How require Resolves Modules

Core module?

e.g. require('fs') → loads built-in

Relative path?

e.g. require('./math') → resolves to file

Package name?

e.g. require('express') → looks in node_modules/

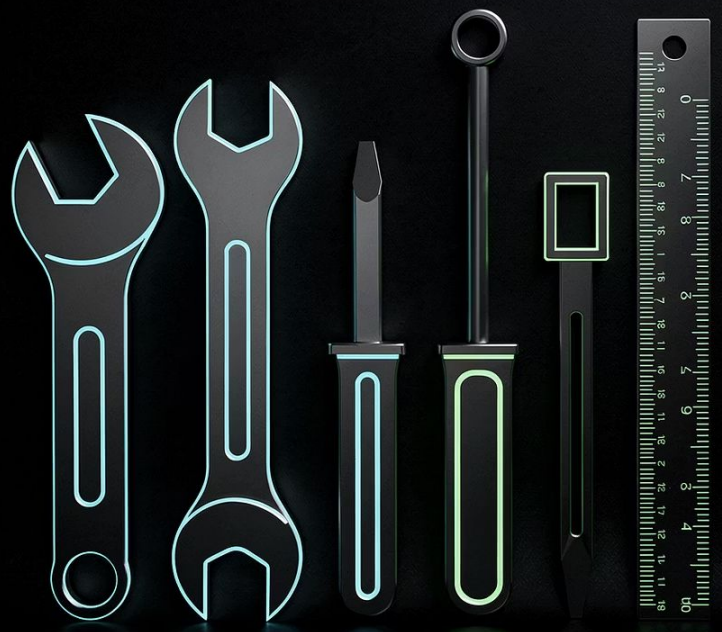
ES Modules (Modern Alternative)

Node also supports ES Modules. To use them, either name your file .mjs or add "type": "module" to package.json:

```
// math.mjs  (or .js with "type": "module" in package.json)
export function add(a, b) {
  return a + b;
}

// app.mjs
import { add } from './math.mjs';
console.log(add(2, 3)); // 5
```

⚠ ⚠️ CommonJS (require) and ES Modules (import) cannot be freely mixed. Stick to one system per project, or use .cjs / .mjs extensions to be explicit.



05

Built-in Node Modules

Built-in Node Modules

No installation needed — these ship with Node:

Module	Purpose
`fs`	File system: read, write, delete files
`path`	Path string utilities
`os`	System info: CPU, memory, platform
`http`	Build HTTP servers
`events`	EventEmitter base class
`util`	Utility helpers (promisify, inspect, etc.)
`crypto`	Hashing, encryption
`stream`	Streaming data (readable, writable, transform)
`url`	Parse and format URLs
`child_process`	Spawn subprocesses

path Examples

```
const path = require("path");

const file = "/users/wesam/projects/app/index.js";

console.log(path.basename(file));       // index.js
console.log(path.dirname(file));        // /users/wesam/projects/app
console.log(path.extname(file));        // .js
console.log(path.join("src", "utils", "helper.js")); // src/utils/helper.js
console.log(path.resolve("src", "index.js"));        // absolute path

// __dirname is the directory of the current file
console.log(__dirname);                 // e.g. /users/wesam/projects/app
console.log(path.join(__dirname, "tasks.json")); // safe absolute path to tasks.json
```



💡 Always use path.join(__dirname, filename) when referencing files relative to your script. Relying on bare filenames breaks when Node is run from a different working directory.

os Examples

```
const os = require("os");

console.log(os.platform());    // 'linux', 'darwin', 'win32'
console.log(os.arch());        // 'x64', 'arm64'
console.log(os.cpus().length); // number of CPU cores
console.log(os.freemem());     // free memory in bytes
console.log(os.homedir());     // /home/wesam
```



06

Blocking vs Non-Blocking Code

Blocking (Synchronous)

This is the most important concept in Node.js. Because Node runs on a single thread, any blocking operation freezes the entire process for all users.

```
const fs = require("fs");

console.log("Before read");

const data = fs.readFileSync("data.txt", "utf8"); // BLOCKS here until file is fully read
console.log(data);

console.log("After read");

// Output (always in this order):
// Before read
// <file contents>
// After read
```

The JS thread is stuck waiting for the OS to read the file. In a server, this would freeze handling of all other requests.

Non-Blocking (Asynchronous)

```
const fs = require("fs");

console.log("Before read");

fs.readFile("data.txt", "utf8", (err, data) => {
  // This callback is called LATER, when the file is ready
  if (err) {
    console.error("Error reading file:", err.message);
    return;
  }
  console.log(data);
});

console.log("After read");  // This runs BEFORE the file is done!

// Output:
// Before read
// After read
// <file contents>
```

Node delegates the file read to libuv/OS, immediately continues executing, and calls the callback when the data is ready. The JS thread is never blocked.

When Is It OK to Use Sync APIs?

Synchronous APIs are acceptable in specific situations:

- CLI scripts that run once and exit (e.g. your task CLI today)
- Application startup — reading config files before the server starts accepting requests
- Build tools and one-off scripts

⊗ **Never use sync APIs inside request handlers or event callbacks in a running server.**



07

The Callback Pattern

The Callback Pattern: Reading Files

Callbacks are the original async pattern in Node. The convention is error-first callbacks: function(err, result).

Standard Pattern

```
fs.readFile("data.txt", "utf8", (err, data) => {
  if (err) {
    console.error("Failed to read:", err.message);
    return; // Always return after handling the error!
  }
  console.log("File contents:", data);
});
```

Reading Files

```
const fs = require("fs");
const path = require("path");

// Always use path.join for safe file paths
const filePath = path.join(__dirname, "data.txt");

fs.readFile(filePath, "utf8", (err, data) => {
  if (err) {
    if (err.code === "ENOENT") {
      console.log("File does not exist yet");
    } else {
      console.error("Unexpected error:", err.message);
    }
    return;
  }
  console.log(data);
});
```

Writing & Appending to Files

Writing Files

```
const fs = require("fs");

fs.writeFile("note.txt", "Hello, Node!", "utf8", (err) => {
  if (err) {
    console.error("Write failed:", err.message);
    return;
  }
  console.log("File written successfully");
});
```

Appending to Files

```
fs.appendFile("log.txt", "New log entry\n", "utf8", (err)
=> {
  if (err) console.error(err.message);
});
```

Callback Hell (and Why We Solve It Later)

Nested callbacks become unreadable quickly:

```
fs.readFile("config.json", "utf8", (err, config) => {
  if (err) return console.error(err);
  fs.readFile("users.json", "utf8", (err, users) => {
    if (err) return console.error(err);
    fs.writeFile("output.json", process(config, users), (err) => {
      if (err) return console.error(err);
      console.log("Done"); // Three levels deep and growing...
    });
  });
});
```

We solve this with Promises and async/await on Day 3.

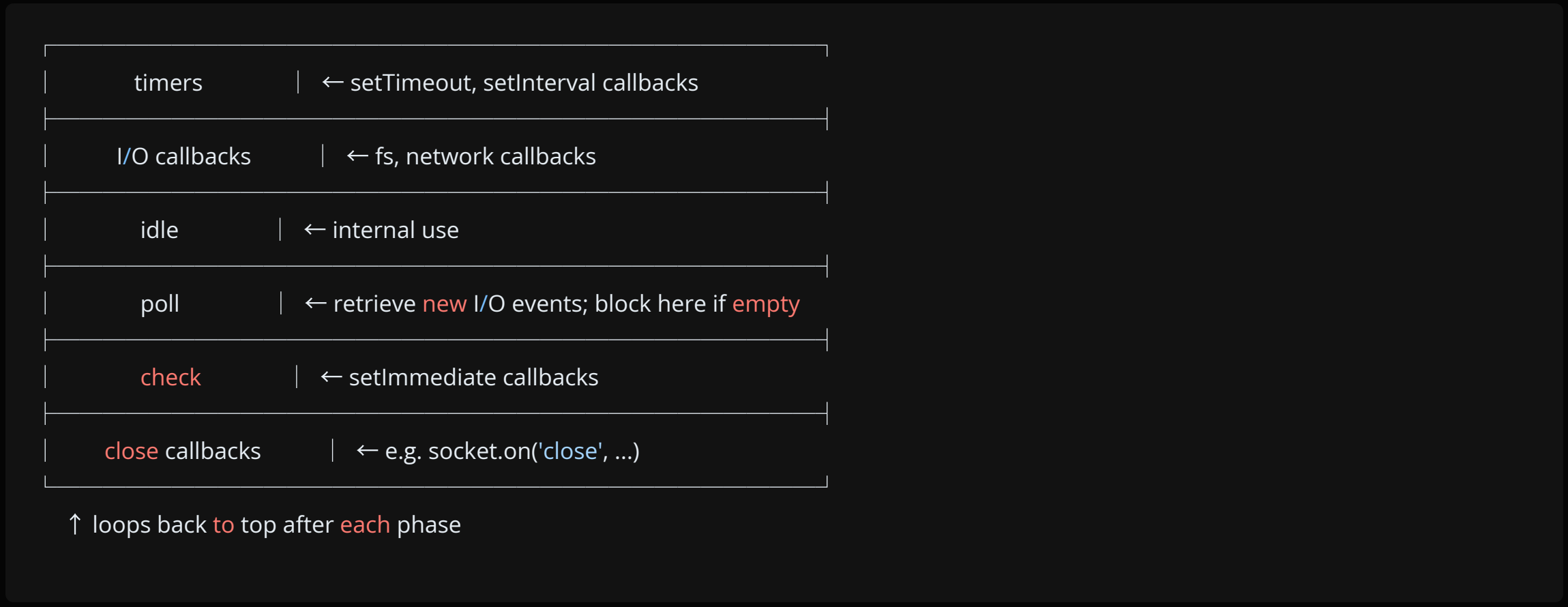


08

The Event Loop

Phases of the Event Loop (simplified)

The event loop is what lets Node handle many concurrent operations with a single thread.



Between each phase: microtasks run (Promise .then() callbacks, process.nextTick()).

Event Loop: Live Example

```
console.log("1 - Start");          // synchronous → call stack

setTimeout(() => {
  console.log("4 - Timer done");   // macrotask → timer phase
}, 0);

Promise.resolve().then(() => {
  console.log("3 - Promise");      // microtask → runs before next event loop phase
});

console.log("2 - End");            // synchronous → call stack

// Output:
// 1 - Start
// 2 - End
// 3 - Promise       ← microtasks run BEFORE timers
// 4 - Timer done
```

Key insight: Even setTimeout(fn, 0) doesn't run immediately — it waits for the call stack to be empty AND for the timer phase of the event loop.

process.nextTick vs Promise.then vs setTimeout

```
setTimeout(() => console.log("setTimeout"), 0);
Promise.resolve().then(() => console.log("Promise.then"));
process.nextTick(() => console.log("nextTick"));
console.log("synchronous");

// Output:
// synchronous
// nextTick       ← runs at end of current operation, before I/O
// Promise.then   ← microtask queue
// setTimeout     ← timer phase (next loop iteration)
```

📌 Further reading: An Animated Guide to Node.js Event Loop



09

NVM — Node Version Manager

Installing & Using NVM

NVM lets you install and switch between multiple Node.js versions — essential when working across projects with different requirements.

Install NVM (macOS / Linux)

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

Restart your terminal (or source ~/.zshrc / source ~/.bashrc), then verify:

```
nvm --version
```

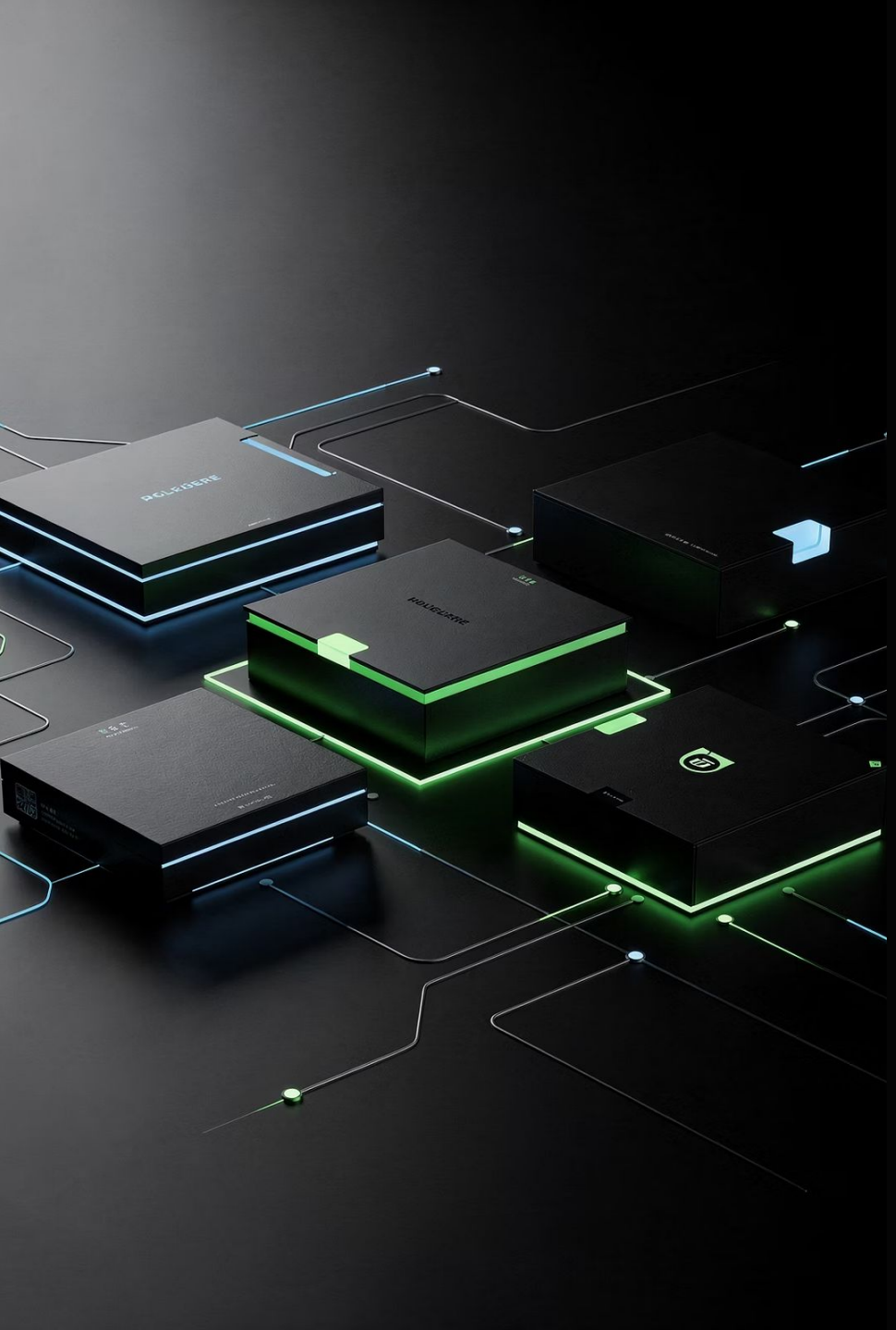
Common NVM Commands

```
nvm install --lts        # Install the latest LTS version (recommended)
nvm install 20           # Install a specific major version
nvm use 20               # Switch to Node 20 in current shell
nvm alias default 20     # Set Node 20 as default for new shells
nvm ls                   # List all installed versions
nvm ls-remote            # List all available versions to install
nvm current              # Show currently active version
```

Why Use NVM?

- Different projects may require different Node versions
- Avoids polluting global system Node installation
- Makes version upgrades easy and reversible
- Aligns your dev environment with production constraints

Windows users: Use nvm-windows instead — it's a separate tool with a very similar interface.



10

NPM — Node Package Manager

Initialize a Project

NPM is Node's built-in package manager. It handles dependencies, scripts, and project metadata.

```
mkdir task-cli
cd task-cli
npm init -y      # -y accepts all defaults
```

This creates package.json:

```
{
  "name": "task-cli",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Installing Packages

```
npm install chalk              # production dependency
npm install nodemon --save-dev # development-only dependency
npm install                    # install all deps listed in package.json
```

This creates:

- node_modules/ — the actual package code (never commit this to git)
- package-lock.json — exact version lockfile (do commit this)

NPM Scripts & .gitignore

NPM Scripts

Define reusable commands in package.json:

```
{
  "name": "task-cli",
  "version": "1.0.0",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "node test.js"
  }
}
```

```
npm run start
npm run dev
npm test        # "test" and "start" don't need the "run" keyword
```

- Standardised entry points for any project
- Abstracts the actual command behind a memorable name
- Can chain multiple commands with && or use tools like npm-run-all

.gitignore — Don't Commit node_modules

Create a .gitignore file in your project root:

```
node_modules/
tasks.json
.env
```

Quick Debugging Reference

Problem	Debug Command
What are my CLI args?	`console.log(process.argv);`
What's in my JSON file?	`cat tasks.json`
Is my file being found?	`console.log(TASKS_FILE);`
Why is my callback not running?	Check if you're calling the async function at all
Unexpected undefined?	Log the full object before accessing properties.

End of Day 1 — Node.js Fundamentals and the Runtime

Bridge to Day 2: You've built a local CLI tool. Tomorrow, we expose it to the outside world by turning it into an HTTP server — and learn how the web actually works.